Primitive Data Types

Integer

In [1]:
```
x = 150
y = 30

print(x)
```

```
150
```

In [2]:
```
x = 150
y = 30

print(type(x))
```

```
<class 'int'>
```

Float

In [3]:
```
x = 1.10
y = 1.0
z = -35.59

print(type(z))
print(z)
```

```
<class 'float'>
-35.59
```

Complex

In [4]:
```
z = 3 + 4j
print(z)
print(z.real)
print(z.imag)
```

```
(3+4j)
3.0
4.0
```

String

In [5]:
```
a = "Hello world"

print(a)
print(type(a))
```

```
Hello world
<class 'str'>
```

Boolean

In [6]:
```
a = True
b = False
print(a)
print(b)
```

```
True
False
```

Non-Primitive Data Types

List

Membuat List

In [7]:
```
huruf = ["a", "b", "c", "d", "e", "f"]

print(huruf[1:4])
```

```
['b', 'c', 'd']
```

In [8]:
```
numbers = [1, 2, 3, 4, 5]

mixed = [10, "Python", 3.14, True]

print(numbers)
print(mixed)
```

```
[1, 2, 3, 4, 5]
[10, 'Python', 3.14, True]
```

Mnegakses Elemen List

In [9]:
```
fruits = ["Apple", "Banana", "Cherry"]

print(fruits[0])
print(fruits[1])
print(fruits[-1])
```

```
Apple
Banana
Cherry
```

Mengubah Elemen List

In [10]:
```
fruits[1] = "Mango"
print(fruits)
```

```
['Apple', 'Mango', 'Cherry']
```

Menambahkan Elemen ke List

In [11]:
```
fruits.append("Orange")
print(fruits)

fruits.insert(1, "Grapes")
print(fruits)
```

```
['Apple', 'Mango', 'Cherry', 'Orange']
['Apple', 'Grapes', 'Mango', 'Cherry', 'Orange']
```

Menghapus Elemen List

In [12]:
```
fruits.remove("Mango")
print(fruits)

fruits.pop(2)
print(fruits)

del fruits[1]
print(fruits)
```

```
['Apple', 'Grapes', 'Cherry', 'Orange']
['Apple', 'Grapes', 'Orange']
['Apple', 'Orange']
```

Menggunakan Loop pada List

In [13]:
```
for fruit in fruits:
    print(fruit)
```

```
Apple
Orange
```

Kesimpulan:

1. List menyimpan banyak nilai dalam satu variabel
2. Bisa diakses, diubah, ditambah, dan dihapus
3. Loop dan List Comprehension membuatnya lebih fleksibel

Tuple

In [14]:
```
numbers = (1, 2, 3, 4, 5)

mixed = (10, "Python", 3.14, True)

print(numbers)
print(mixed)
```

```
(1, 2, 3, 4, 5)
(10, 'Python', 3.14, True)
```

Jika hanya satu elemen dalam tuple, tambahkan koma , agar Python mengenali sebagai tuple.

In [15]:
```
single_element = (5,)
not_tuple = (5)

print(type(single_element))
print(type(not_tuple))
```

```
<class 'tuple'>
<class 'int'>
```

Mengakses Elemen Tuple

In [16]:
```
fruits = ("Apple", "Banana", "Cherry")

print(fruits[0])
print(fruits[-1])
```

```
Apple
Cherry
```

Tuple tidak bisa diubah jika sudah dibuat, contoh:

tidak seperti list yang bisa diubah

In [17]:
```
fruits[1] = "Mango"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_295/2126348466.py in <cell line: 0>()
----> 1 fruits[1] = "Mango"

TypeError: 'tuple' object does not support item assignment
```

Bisa Menyimpan Banyak Nilai Sekaligus (Packing & Unpacking)

In [19]:
```
data = ("Ezha", 21, "Data Scientist")

name, age, job = data

print(name)
print(age)
print(job)
```

```
Ezha
21
Data Scientist
```

Menggabungkan Tuple

In [20]:
```
tuple1 = (1, 2, 3)
tuple2 = (4, 5, 6)
```

```
new_tuple = tuple1 + tuple2
print(new_tuple)
```

```
(1, 2, 3, 4, 5, 6)
```

Menghitung Elemen dalam Tuple

In [21]:
```
numbers = (1, 2, 2, 3, 4, 2, 5)

print(numbers.count(2))
print(numbers.index(5))
```

```
3
6
```

Kesimpulan:

1. Tuple mirip list, tetapi immutable (tidak bisa diubah).
2. Lebih cepat dan aman dibanding list jika datanya tetap.
3. Bisa digunakan untuk menyimpan data yang tidak boleh berubah.

Hands On

In [22]:
```
lokasi = (12.5, -7.8, 100.2)

print("Koordinat awal:", lokasi)
```

```
Koordinat awal: (12.5, -7.8, 100.2)
```

In [23]:
```
lokasi[0] = 15.0  # Apakah ini berhasil?
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_295/621390748.py in <cell line: 0>()
----> 1 lokasi[0] = 15.0  # Apakah ini berhasil?

TypeError: 'tuple' object does not support item assignment
```

Dictionary

Membuat Dictionary

In [24]:
```
student = {
    "nama": "Ali Muhsin",
    "umur": 21,
    "jurusan": "Artificial Intelligence"
}

print(student)
```

```
{'nama': 'Ali Muhsin', 'umur': 21, 'jurusan': 'Artificial Intelligence'}
```

Dictionary menggunakan kurung kurawal {}.

Mengakses Nilai dalam Dictionary

In [25]:
```
print(student["nama"])
print(student["umur"])
```

```
Ali Muhsin
21
```

Menambahkan atau Mengubah Data dalam Dictionary

In [26]:
```
student["alamat"] = "Yogyakarta"

student["umur"] = 25

print(student)
```

```
{'nama': 'Ali Muhsin', 'umur': 25, 'jurusan': 'Artificial Intelligence', 'alamat':
'Yogyakarta'}
```

menghapus elemen dalam dictionary

In [27]:
```
del student["jurusan"]

umur = student.pop("umur")

print(student)
print(umur)
```

```
{'nama': 'Ali Muhsin', 'alamat': 'Yogyakarta'}
25
```

Fungsi pop() pada dictionary digunakan untuk menghapus sebuah key dan mengembalikan nilai (value) yang dihapus. sedangkan "del" hanya menghapus tanpa menyimpan nilai

Set

In [28]:
```
angka = {1, 2, 3, 2, 4, 5}

huruf = {"a", "b", "c", "d"}

campuran = {1, "Python", 3.14}

print(angka)
print(huruf)
print(campuran)
```

```
{1, 2, 3, 4, 5}
{'c', 'b', 'a', 'd'}
{1, 3.14, 'Python'}
```

Ciri-ciri set:

1. Tidak ada elemen yang duplikat
2. Tidak memiliki indeks (tidak bisa diakses dengan angka index seperti list)

Menghapus Duplikat dari List dengan Set

In [29]:
```
angka_list = [1, 2, 2, 3, 4, 4, 5]
angka_set = set(angka_list)

print(angka_set)
```

```
{1, 2, 3, 4, 5}
```

Menambah dan Menghapus Elemen Menggunakan Set

In [30]:
```
angka.add(6)
angka.remove(3)
print(angka)
```

```
{1, 2, 4, 5, 6}
```

Operasi Himpunan (set operations) Irisan (Intersection)

In [31]:
```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a & b)
```

```
{3, 4}
```

Gabungan atau Union

In [32]:
```
print(a | b)
```

```
{1, 2, 3, 4, 5, 6}
```

Selisih (Difference)

In [33]:
```
print(a - b)
print(b - a)
```

```
{1, 2}
{5, 6}
```

Kesimpulan

1. set digunakan untuk menyimpan elemen unik
2. Tidak memiliki indeks atau urutan tetap
3. Cocok untuk operasi himpunan (union, intersection, dll.)
4. Berguna untuk menghapus duplikat dari list

In [34]:
```
id_pengguna = {101, 202, 303, 404}

print("Sebelum ditambah:", id_pengguna)
```

```
Sebelum ditambah: {202, 404, 101, 303}
```

Hands On

Jika saya ingin menambahkan

1. id_pengguna.add(505) dan
2. id_pengguna.add(101) apa yang terjadi?

In [35]:
```
id_pengguna.add(505)
print("Setelah ditambah:", id_pengguna)
```

```
Setelah ditambah: {101, 202, 303, 404, 505}
```

In [36]:
```
id_pengguna.add(101)
print("Setelah ditambah:", id_pengguna)
```

```
Setelah ditambah: {101, 202, 303, 404, 505}
```

Konversi Tipe Data

Konversi Otomatis (Implicit Conversion)

In [37]:
```
angka_int = 10
angka_float = angka_int + 3.5

print(angka_float)
print(type(angka_float))
```

```
13.5
<class 'float'>
```

Python mengubah 10 (int) menjadi 10.0 (float) agar sesuai dengan operasi float.

Konversi Manual (Explicit Conversion)

int ke float / float ke int

In [38]:
```
x = 5
x_float = float(x)

y = 7.8
y_int = int(y)

print(x_float)
print(y_int)
```

```
5.0
7
```

int/float ke string

In [39]:
```
angka = 42
angka_str = str(angka)

teks = "100"
```

```
teks_int = int(teks)

print(angka_str, type(angka_str))
print(teks_int, type(teks_int))
```

```
42 <class 'str'>
100 <class 'int'>
```

list ke tuple

In [40]:
```
data_list = [1, 2, 3]
data_tuple = tuple(data_list)
data_tuple2 = (4, 5, 6)
data_list2 = list(data_tuple2)

print(data_tuple, type(data_tuple))
print(data_list2, type(data_list2))
```

```
(1, 2, 3) <class 'tuple'>
[4, 5, 6] <class 'list'>
```

list/tuple ke set

In [41]:
```
angka_list = [1, 2, 2, 3]
angka_set = set(angka_list)

angka_tuple = tuple(angka_set)

print(angka_set)
print(angka_tuple)
```

```
{1, 2, 3}
(1, 2, 3)
```

Dictionary ke Lis

In [42]:
```
data_dict = {"nama": "Riccy", "umur": 17}

keys_list = list(data_dict.keys())

values_list = list(data_dict.values())

print(keys_list)
print(values_list)
```

```
['nama', 'umur']
['Riccy', 17]
```

Kesimpulan:

1. Konversi Otomatis terjadi saat Python perlu menyamakan tipe data dalam operasi.
2. Konversi Manual (Casting) dilakukan dengan fungsi bawaan (int(), float(), str(), list(), dll.).
3. Gunakan konversi yang tepat sesuai kebutuhan, terutama untuk menghindari error saat operasi antar tipe data.